

SkyLink Airways — Design Report

1. Introduction

The system is a console-based prototype for SkyLink Airways. It replaces spreadsheet-style booking management with an object-oriented application for flights, passengers and tickets. The design follows the assignment requirements by separating domain objects from menu interaction and by using inheritance and runtime polymorphism.

2. Class and OOP Design

`Flight` is an abstract base class because all flight types share common information but use different fare rules. `DomesticFlight`, `InternationalFlight`, and `CharterFlight` override virtual functions. This demonstrates abstraction, inheritance and runtime polymorphism.

`Passenger` is also abstract. Economy, Business and First Class passengers provide different baggage allowances, loyalty multipliers and cancellation percentages. The cancellation operation accesses these virtual rules through a base-class reference, so the correct passenger-specific behavior is selected at runtime.

`Ticket` represents a booking between a passenger and a flight. It stores a unique ticket ID, passenger ID, flight number, seat number, fare and booking status.

`Airline` owns the flight and passenger objects using `std::unique_ptr` and stores tickets in a `std::vector`. Flights and passengers are indexed by ID using `std::map`. This provides automatic memory management and efficient lookup.

3. Required C++ Features

Operator `<<` is overloaded for both `Flight` and `Ticket` so objects can be printed naturally. `Ticket ==` compares tickets by ID.

The generic `genericSearch` function demonstrates a reusable function template. STL algorithms including `std::find_if`, `std::any_of`, and `std::sort` are used for searching, validation and revenue ranking.

Custom exceptions include `FlightFullException`, `DuplicateBookingException`, `InvalidCancellationException`, and `NotFoundException`. The menu catches standard exceptions so invalid operations do not terminate the application.

4. Booking and Refund Logic

Booking checks that the passenger exists, the flight exists, and the passenger does not already have an active ticket for that flight. The flight reserves a seat before the ticket is stored.

Cancellation marks the ticket as cancelled and releases its seat. The refund is based on the passenger's cancellation percentage and is reduced when the cancellation occurs close to departure. The implementation uses three time bands: more than 24 hours, 6–24 hours, and less than 6 hours.

5. Persistence

The system stores flights, passengers, tickets and the next ticket number in a text file. On startup, the program attempts to load the file. On exit, it saves the current state. This allows the application to remember state between runs.

6. Reports

Three report functions are provided:

- Today's departures by date.
- Occupancy percentage for every flight.
- Top five active-ticket revenue flights.

Revenue is accumulated from active tickets and sorted using ``std::sort``.

7. Testing

8. Limitations

This prototype uses simplified pricing and refund rules. It does not implement authentication, payment gateways, graphical UI, real airline APIs, or a relational database. Date/time processing is intentionally lightweight and is based on the assignment's console prototype scope.

9. Conclusion

The design demonstrates the requested OOP concepts while keeping responsibilities separated. The class hierarchy makes it possible to add new flight or passenger types later without rewriting the main booking architecture.

Test cases

Test	Expected result
List seeded flights	10 flights displayed
List seeded passengers	8 passengers displayed
Book a free seat	Ticket created
Duplicate booking	DuplicateBookingException
Full flight	FlightFullException
Cancel active ticket	Cancelled; seat released
Cancel cancelled ticket	InvalidCancellationException
Search	Matching records displayed
Reports	Occupancy and revenue results displayed
Save/restart	State restored